

Analyze code using SonarQube and generate reports

AIM

Analyze code using SonarQube and generate reports.

SOFTWARE REQUIRED

- Windows operating system
- SonarQube Community Build
- Java JDK
- Python
- SonarQube Scanner for Python (PySonar)
- Web browser such as Google Chrome

THEORY

SonarQube is an open-source platform used for **continuous inspection and analysis of source code**. It helps developers identify problems in software related to **code quality, reliability, security, and maintainability**. SonarQube supports multiple programming languages and provides a centralized web-based dashboard for viewing analysis results.

SonarQube works by analyzing the source code using a collection of predefined **coding rules**. A scanner is used to inspect the project and send the analysis results to the SonarQube server. The server processes these results and presents them through the SonarQube dashboard.

The major aspects analyzed by SonarQube are:

1. **Bugs** – Detects coding errors that may cause incorrect or unexpected program behavior.
2. **Vulnerabilities** – Identifies security weaknesses that may be exploited by attackers.
3. **Code Smells** – Detects code structures that may make the software difficult to understand, maintain, or modify.

4. **Security Hotspots** – Identifies security-sensitive sections of code that require manual review.
5. **Code Coverage** – Indicates the percentage of source code that is executed by automated tests.
6. **Code Duplication** – Detects repeated or duplicated sections of code, which can increase maintenance effort.

ADDITIONAL LEARNING

Through this experiment, the following additional concepts were learned:

1. **Continuous Code Inspection**: SonarQube can continuously monitor source code and identify quality issues during software development.
2. **Static Code Analysis**: SonarQube analyzes source code without executing the program, helping detect potential bugs and security problems early.
3. **Quality Gates**: Quality Gates provide a measurable standard for deciding whether code is ready for further development or deployment.
4. **Technical Debt**: Code smells and maintainability issues can increase technical debt. SonarQube helps identify such areas so they can be improved.
5. **Security Analysis**: SonarQube helps detect vulnerabilities and security-sensitive code that may require further review.

CONCLUSION

SonarQube was successfully used to perform static code analysis and generate a detailed report of the source code. The experiment demonstrated how SonarQube identifies bugs, vulnerabilities, code smells, security hotspots, code duplication, and other maintainability issues. The Quality Gate provided an overall assessment of the project's code quality. Thus, SonarQube is an effective tool for improving software quality, security, reliability, and maintainability and helps developers detect and resolve problems at an early stage.

